



University for Business and Technology

MSc – Computer Science and Engineering

2024/2025

Stilet e Arkitekturës dhe Mostrat e dizajnit

-Sistem për menaxhimin e një librarie-

Studenti: Erind Avdiu

Profesori: Ramadan Dervishi

Shkurt, 2026.

Përmbajtja

1. Përshkrimi i projektit dhe objektivat	3
2. Kërkesat kryesore.....	4
3. Diagrami i rasteve të përdorimit (use case diagram)	6
4. Diagrami i klasave (class diagram).....	8
5. Diagramet e Sekuencës (sequence diagrams)	10
6. Diagrami i Aktivitetit (activity diagram).....	13
7. Arkitektura dhe parimet e dizajnit.....	15
7. Implementimi dhe përdorimi	19
8. Konkluzioni	20

1. Përshkrimi i projektit dhe objektivat

1.1 Përshkrimi i Projektit

Sistemi i Menaxhimit të Bibliotekës është një aplikacion backend i zhvilluar me **Fastify** dhe TypeScript, i ndërtuar për të demonstruar në mënyrë praktike dhe të strukturuar parimet e avancuara të Programimit të Orientuar në Objekte (OOP) dhe parimet e dizajnit SOLID. Projekti fokusohet në ndërtimin e një API-je të qëndrueshme, të tipizuar dhe të zgjerueshme, e cila simulon funksionimin real të një sistemi bibliotekar për menaxhimin e resurseve dhe proceseve të huazimit.

Përmes përdorimit të TypeScript, sistemi përfiton nga tipizimi statik, kontratat përmes interface-ve dhe kontrolli më i mirë i varësive mes moduleve. Projekti gjithashtu demonstroi përdorimin e pattern-eve të zakonshme të dizajnit si Strategy, Repository dhe Dependency Injection për të ndërtuar një bazë të fortë arkitekturore. Në këtë mënyrë, aplikacioni shërben jo vetëm si zgjidhje funksionale, por edhe si shembull edukativ për ndërtimin e sistemeve backend të shkallëzueshme.

1.2 Objektivat dhe qëllimi i projektit

Qëllimi kryesor i projektit është të demonstrojë në mënyrë të plotë dhe të argumentuar zbatimin praktik të parimeve të OOP dhe SOLID në një aplikacion real backend, duke treguar se si konceptet teorike si abstraksioni, trashëgimia, polimorfizmi dhe përdorimi i interface-ve mund të shndërrohen në zgjidhje konkrete për strukturimin e një sistemi funksional me rregulla të qarta biznesi

Projekti synon gjithashtu:

- të krijojë një arkitekturë të ndarë në shtresa (controller, service, domain, repository) për ndarje të qartë të përgjegjësiave
- të demonstrojë përdorimin e polimorfizmit përmes strategjive të ndryshme të huazimit të librave
- të përdorë interface dhe klasa abstrakte për të krijuar kontrata të qëndrueshme midis komponentëve

- të mundësojë zgjerim të lehtë të sistemit me tipe të reja anëtarësh ose rregulla të reja huazimi
- të dokumentojë strukturën përmes diagrameve UML për klasat dhe marrëdhëniet midis tyre

2. Kërkesat kryesore

2.1. Kërkesat funksionale

Sistemi duhet të ofrojë funksionalitete bazë për menaxhimin e një biblioteke në nivel backend përmes endpoint-eve API. Çdo funksion realizohet përmes kërkesave HTTP dhe trajtohet nga shtresa e controller-it dhe service-it.

- **Krijimi i librave** – Sistemi duhet të lejojë regjistrimin e një libri të ri duke pranuar të dhëna bazë si titulli, autori dhe ISBN. ISBN duhet të jetë unik dhe të validohet para ruajtjes.
- **Listimi i librave** – Duhet të ekzistojë endpoint për kthimin e listës së të gjithë librave të regjistruar, së bashku me statusin e tyre (i disponueshëm / i huazuar).
- **Krijimi i anëtarëve** – Sistemi duhet të mundësojë regjistrimin e anëtarëve me emër, email dhe lloj anëtari (p.sh. standard, premium, student). Email-i duhet të kontrollohet për format korrekt.
- **Listimi i anëtarëve** – Duhet të ofrohet funksion për marrjen e listës së plotë të anëtarëve ekzistues.
- **Huazimi i librave** – Një anëtar mund të huazojë një libër vetëm nëse libri është i disponueshëm dhe nëse nuk tejkalohet limiti i huazimeve sipas rregullave të strategjisë së zgjedhur. Regjistrohet data e huazimit.
- **Kthimi i librave** – Sistemi duhet të lejojë kthimin e një libri të huazuar dhe të përditësojë statusin e tij në të disponueshëm.
- **Kontrolli i disponueshmërisë** – Para çdo huazimi, sistemi duhet të kontrollojë nëse libri është i lirë dhe nuk është i lidhur me një huazim aktiv.

2.2. Kërkesat teknike

Këto kërkesa përcaktojnë mjedisin dhe mënyrën e implementimit teknik të sistemit.

- **Gjuhë Programimi:** TypeScript 5.4.5 – përdoret për tipizim statik, interface dhe kontrata të qarta mes komponentëve.
- **Framework:** Fastify 5.7.4 – përdoret për ndërtimin e API-ve REST me performancë të lartë dhe strukturë plugin-based.
- **Runtime:** Node.js – ekzekutimi i aplikacionit në server përmes modelit event-driven.
- **Arkitektura:** Clean Architecture – ndarje në shtresa (controllers, services, domain, repositories) ku varësitë drejtohen nga jashtë drejt brendësisë së domenit.
- **Ruajtja e të Dhënave:** Në memorie – përdoren koleksione (arrays/maps) në repository për ruajtje të përkohshme, pa integrim me bazë të dhënash, për të mbajtur fokusin te dizajni dhe OOP.
- **Validimi i input-it:** Kontroll i të dhënave hyrëse në nivel controller/service për të parandaluar gjendje jo valide.
- **Strukturë modulare:** Kodi ndahet në module sipas domenit (books, members, loans).

2.3. Kërkesat e dizajnit

Kërkesat e dizajnit fokusohen në cilësinë arkitekturore dhe mënyrën e strukturimit të kodit.

- **Zbatimi i parimeve OOP** – Entitetet si Book, Member dhe Loan modelohen si klasa me encapsulation të attributeve dhe metodave.
- **Abstraksion dhe klasa abstrakte** – Përdoren klasa abstrakte për tipe të përgjithshme (p.sh. MemberBase) nga të cilat trashëgohen tipe konkrete anëtarësh.
- **Polimorfizëm** – Strategji të ndryshme huazimi implementojnë të njëjtin interface por me sjellje të ndryshme.

- **Interface** – Repository dhe strategjitë definojnë përmes interface-ve për të shkëputur implementimin nga përdorimi.
- **SOLID (S dhe O në fokus)** – Çdo klasë ka përgjegjësi të vetme, ndërsa funksionalitetet e reja shtohen me implementime të reja pa ndryshuar klasat ekzistuese.
- **Dependency Injection** – Shërbimet marrin varësi përmes konstruktorit, jo duke i krijuar vetë, për testim dhe zëvendësim më të lehtë.
- **Dizajn i zgjerueshëm** – Sistemi lejon shtimin e tipeve të reja anëtarësh, rregullave të reja huazimi dhe mënyrave të reja ruajtjeje pa ndryshim të kodit bazë.

Nëse dëshiron, mund ta formatoj këtë seksion direkt në stil raporti/teme diplome që të mbushë saktë disa faqe.

3. Diagrami i rasteve të përdorimit (use case diagram)

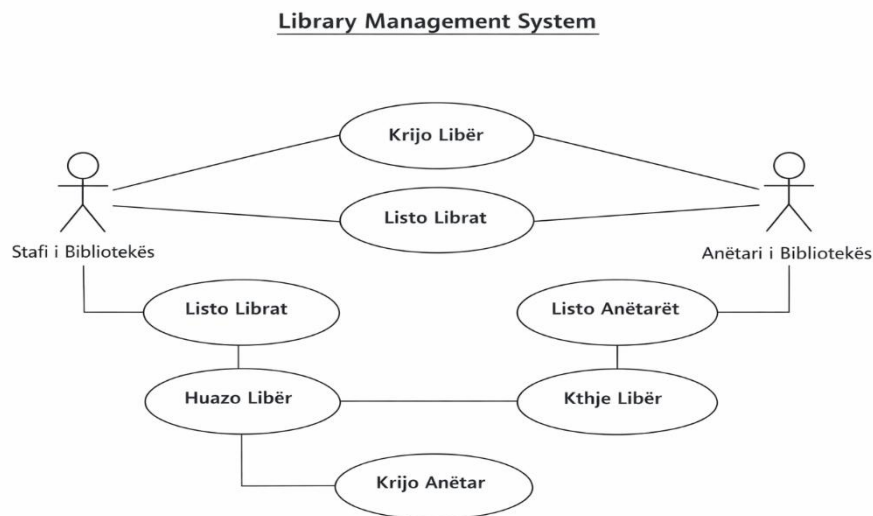


Diagram 1. Use Case Diagram of Libraby Management System

Diagrami i rasteve të përdorimit paraqet ndërveprimin mes aktorëve dhe funksioneve kryesore të Sistemit të Menaxhimit të Bibliotekës. Në këtë diagram identifikohen dy aktorë kryesorë: Stafi i Bibliotekës dhe Anëtari i Bibliotekës, të cilët komunikojnë me sistemin përmes rasteve të ndryshme të përdorimit (use cases). Diagrami tregon qartë kufirin e sistemit dhe funksionalitetet që ofrohen për secilin rol.

Stafi i Bibliotekës ka qasje administrative dhe kryen operacionet e menaxhimit të të dhënave, si krijimi i librave, krijimi i anëtarëve dhe shikimi i listave përkatëse. Ky aktor përfaqëson përdoruesin me privilegje të plota mbi sistemin. Në anën tjetër, Anëtari i Bibliotekës ka rol përdoruesi funksional, i fokusuar në operacionet e huazimit dhe kthimit të librave, si dhe në shikimin e katalogut të librave.

Rastet e përdorimit të paraqitura në diagram përfshijnë: krijimin e librit, listimin e librave, krijimin e anëtarit, listimin e anëtarëve, huazimin e librit dhe kthimin e librit. Këto raste mbulojnë rrjedhën bazë të proceseve të bibliotekës dhe përfaqësojnë funksionet minimale të nevojshme për funksionimin e sistemit backend. Diagrami shërben si përmbledhje vizuale e kërkesave funksionale dhe si bazë për modelimin e mëtejshëm të klasave dhe shërbimeve në arkitekturë.

4. Diagrami i klasave (class diagram)

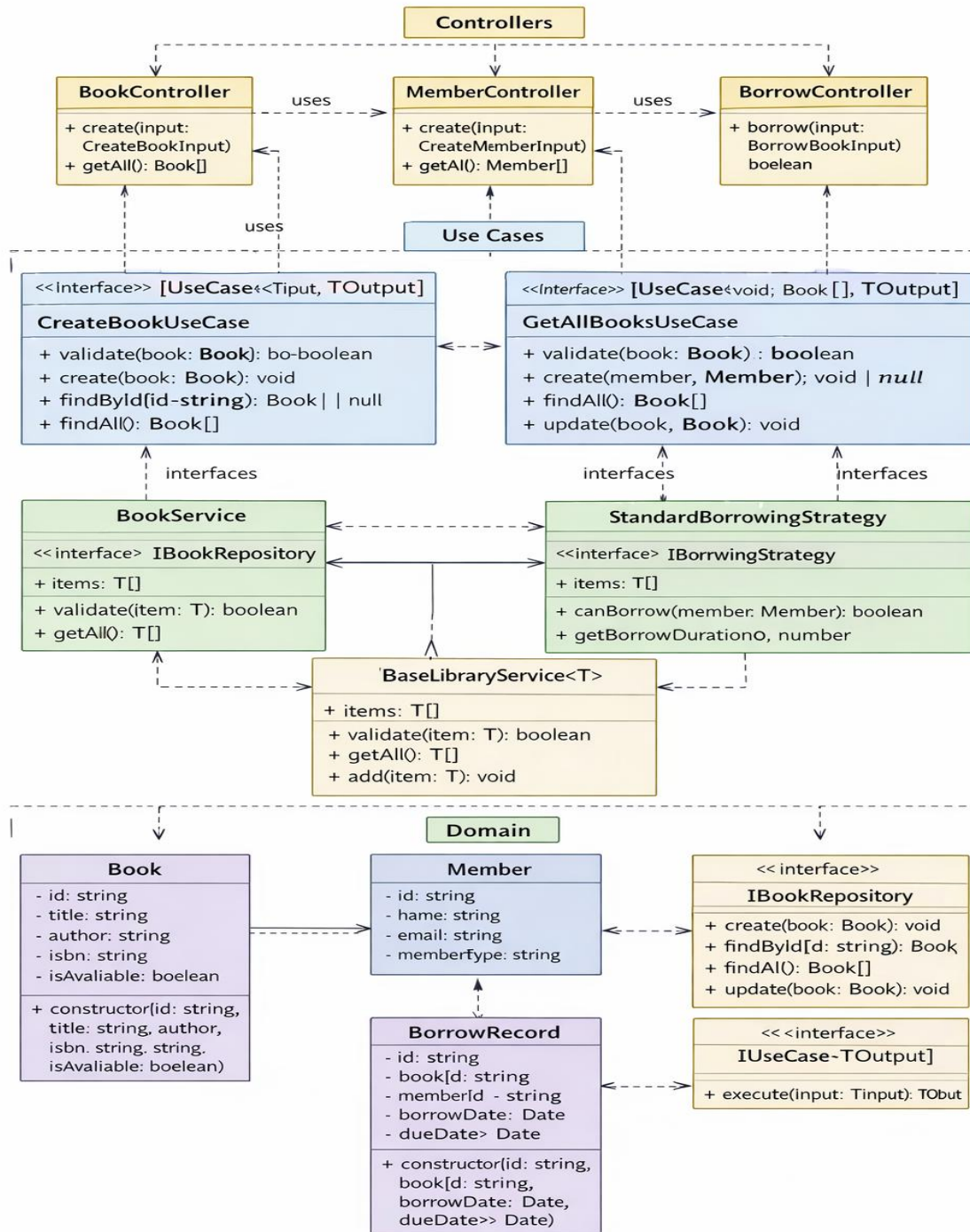


Diagram 2. Class Diagram of System.

Diagrami i klasave i Sistemit të Menaxhimit të Bibliotekës përfaqëson strukturën e plotë të aplikacionit duke ndarë qartë komponentët sipas shtresave: Domain, Services, Use Cases dhe Controllers. Ai tregon marrëdhëniet midis klasave, interface-ve dhe klasave abstrakte, si dhe lidhjet e trashëgimisë dhe përdorimit të varësive.

Në Shtresën Domain ndodhen entitetet kryesore: Book, Member dhe BorrowRecord, të cilat përfaqësojnë të dhënat bazë të sistemit. Secili entitet ka atributet dhe konstruktorët e nevojshëm për inicializimin e tyre. Në këtë nivel gjenden edhe interface-t IBookRepository, IMemberRepository dhe IBorrowingStrategy që sigurojnë kontratat për shërbimet dhe strategjitë e huazimit, si dhe interface-i i përgjithshëm IUseCase<TInput, TOutput> për rastet e përdorimit.

Shtresa e Services përfshin klasat që menaxhojnë logjikën e biznesit dhe qasjen në të dhëna. BookService dhe MemberService trashëgojnë nga klasa abstrakte BaseLibraryService<T> dhe implementojnë interface-t përkatëse të repository-ve. Strategjitë e huazimit (StandardBorrowingStrategy, PremiumBorrowingStrategy, StudentBorrowingStrategy) implementojnë IBorrowingStrategy dhe përcaktojnë rregullat e veçanta për huazimin e librave.

Shtresa e Use Cases përmban klasat që orkestrojnë logjikën e veprimeve të sistemit, si CreateBookUseCase, GetAllBooksUseCase, BorrowBookUseCase dhe ReturnBookUseCase. Këto implementojnë interface-in IUseCase dhe përdorin shërbimet për të realizuar funksionalitetet e kërkuara.

Shtresa e Controllers përfaqëson hyrjen e sistemit nga përdoruesit. BookController, MemberController dhe BorrowController përdorin rastet e përdorimit për të kryer veprimet e kërkuara nga aktorët (stafi dhe anëtarët).

Diagrami gjithashtu tregon marrëdhëniet kryesore:

- Inheritance (extends) midis BookService / MemberService dhe BaseLibraryService
- Implementation (implements) për interface-t si repository, use case dhe strategjitë

- Dependency (uses) midis controller-ve dhe use case-ve, si dhe midis use case-ve dhe services/strategjive.

Ky diagram siguron një pasqyrë të qartë të arkitekturës së sistemit, duke lidhur të dhënat, logjikën e biznesit dhe ndërfaqen për përdoruesin në një model të qëndrueshëm dhe të zgjerueshëm.

5. Diagramet e Sekuencës (sequence diagrams)

Diagramat e sekuencës ilustrojnë rrjedhën e mesazheve midis objekteve gjatë ekzekutimit të funksioneve kryesore të sistemit: **krijimi i librit** dhe **huazimi i librit**. Ato tregojnë pjesëmarrësit (actors dhe objekte), thirrjet e metodave dhe kthimet e vlerave, duke e bërë të qartë ndërveprimin mes klientit, controller-it, use case-ve dhe services. Kutitë e aktivitetit (activation boxes) tregojnë periudhat gjatë të cilave objektet janë aktive në ekzekutim.

5.1 Krijimi i Librit (Create Book Sequence Diagram)

Ky diagram tregon rrjedhën për krijimin e një libri të ri:

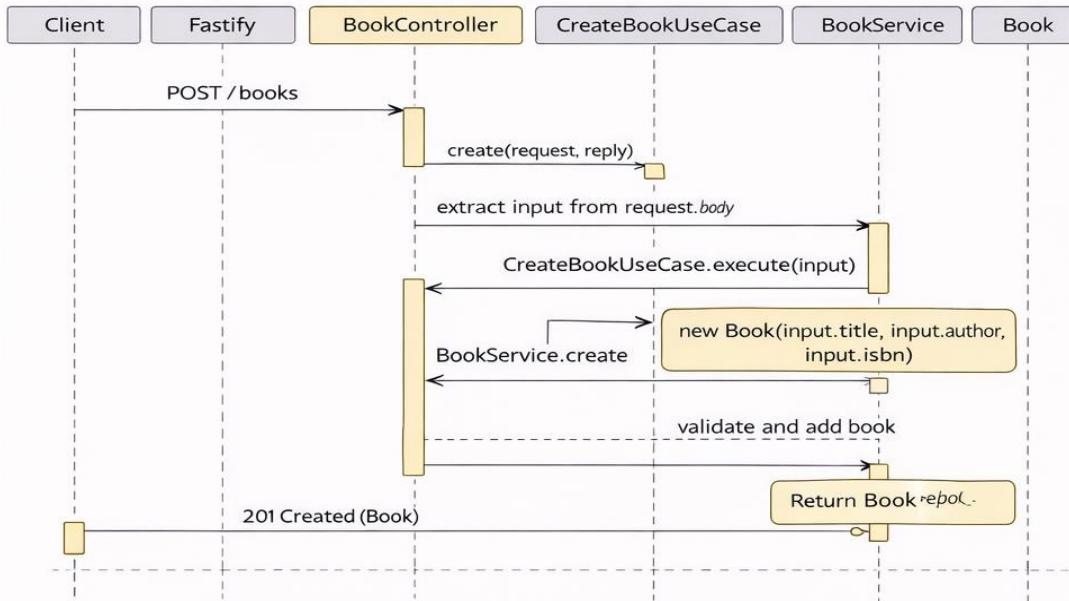
1. Klienti dërgon kërkesë HTTP POST në endpoint /books.
2. Fastify merr kërkesën dhe e transmeton te BookController.
3. BookController nxjerr të dhënat e librave nga request.body.
4. BookController thërret CreateBookUseCase.execute().
5. CreateBookUseCase krijon një objekt Book të ri.
6. CreateBookUseCase thërret BookService.create().
7. BookService validon librin dhe e shton në sistem.
8. BookService kthen objektin Book tek CreateBookUseCase.
9. CreateBookUseCase kthen objektin Book tek BookController.
10. BookController dërgon përgjigje HTTP 201 me librin e krijuar.

5.2 Huazimi i Librit (Borrow Book Sequence Diagram)

Ky diagram tregon rrjedhën e procesit të huazimit të librit me vendime për sukses ose dështim:

1. Klienti dërgon kërkesë HTTP POST në /borrow me bookId dhe memberId.
2. Fastify merr kërkesën dhe e kalon te BorrowController.
3. BorrowController nxjerr të dhënat nga request.body.
4. BorrowController thërret BorrowBookUseCase.execute().
5. BorrowBookUseCase thërret BookService.findById() për të gjetur librin.
6. BorrowBookUseCase thërret MemberService.findById() për të gjetur anëtarin.
7. BorrowBookUseCase thërret IBorrowingStrategy.canBorrow() për të kontrolluar nëse anëtari mund të huazojë librin.
8. Nëse mund të huazohet:
 - Thërret IBorrowingStrategy.getBorrowDuration() për të marrë kohëzgjatjen e huazimit.
 - Përditëson disponueshmërinë e librit në false.
 - Krijon një objekt BorrowRecord me datën e duhur.
 - Kthen BorrowRecord tek BorrowController.
9. BorrowController kthen përgjigje HTTP 201 me BorrowRecord në rast suksesi ose HTTP 400 me gabim në rast dështimi.

Create Book Sequence Diagram



Borrow Book Sequence Diagram

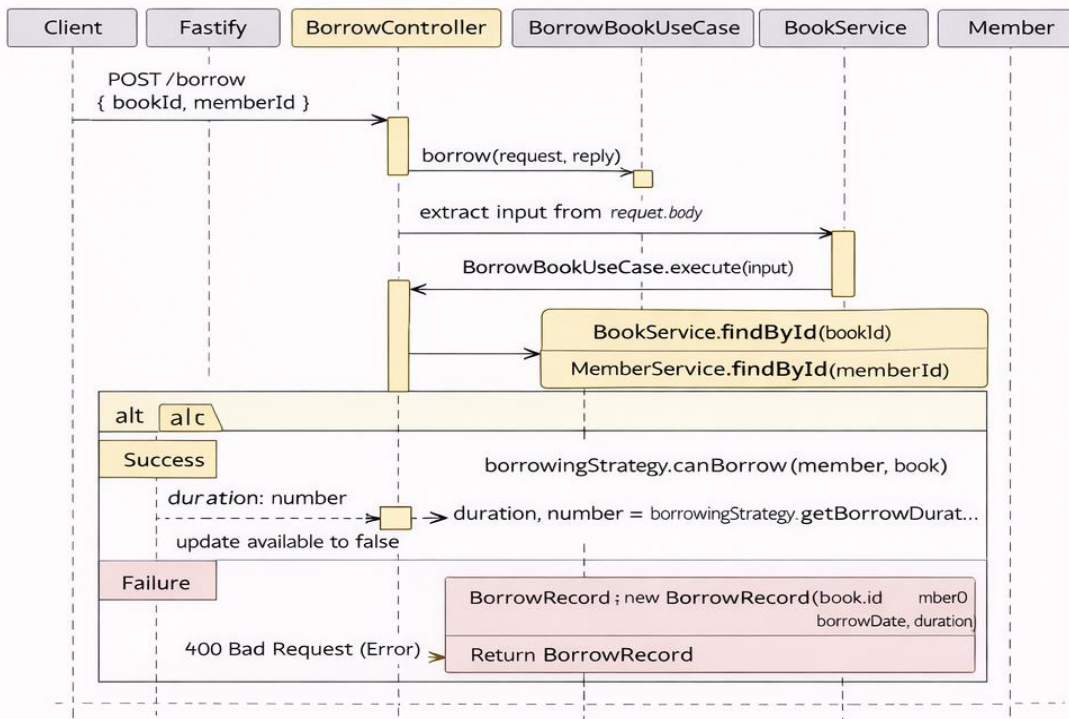


Diagram 3. Diagramet e sekuencës për krijimin e librit dhe huazimin e librit.

6. Diagrami i Aktivitetit (activity diagram)

Diagrami i aktivitetit për procesin e huazimit të librit përfaqëson rrjedhën hap pas hapi nga kërkesa e një anëtari deri tek regjistrimi i suksesit ose dështimit të huazimit. Ai përdor nyja fillestare dhe përfundimtare, aktivitete të shprehura në forma të rrumbullakëta (rounded rectangles), pika vendimi (diamonds) për kontrollin e kushteve, dhe flukse me etiketa “Po”/“Jo”. Opsionalisht mund të përdoren swimlanes për të ndarë përgjegjësitë midis aktorëve dhe sistemit.

Procesi fillon me kërkesën për huazim dhe përfshin hapat: kërkimin e librit, kontrollin e disponueshmërisë, kërkimin e anëtarit dhe verifikimin e të drejtave të huazimit sipas strategjisë së përcaktuar. Në rast se ndonjë kusht nuk plotësohet (libri nuk ekziston, anëtari nuk ekziston, ose strategjia e huazimit e ndalon), procesi përfundon me gabim. Në rast të suksesit, sistemi llogarit kohëzgjatjen e huazimit, përditëson statusin e librit në “i huazuar”, krijon rekord të ri BorrowRecord dhe kthen këtë rekord si rezultat të suksesshëm.

Ky diagram siguron një vizualizim të qartë të rrjedhës logjike dhe vendimeve kritike gjatë huazimit, duke ndihmuar zhvilluesit dhe përdoruesit teknikë të kuptojnë logjikën e sistemit dhe të ndjekin çdo hap të procesit.

Borrow Book Process

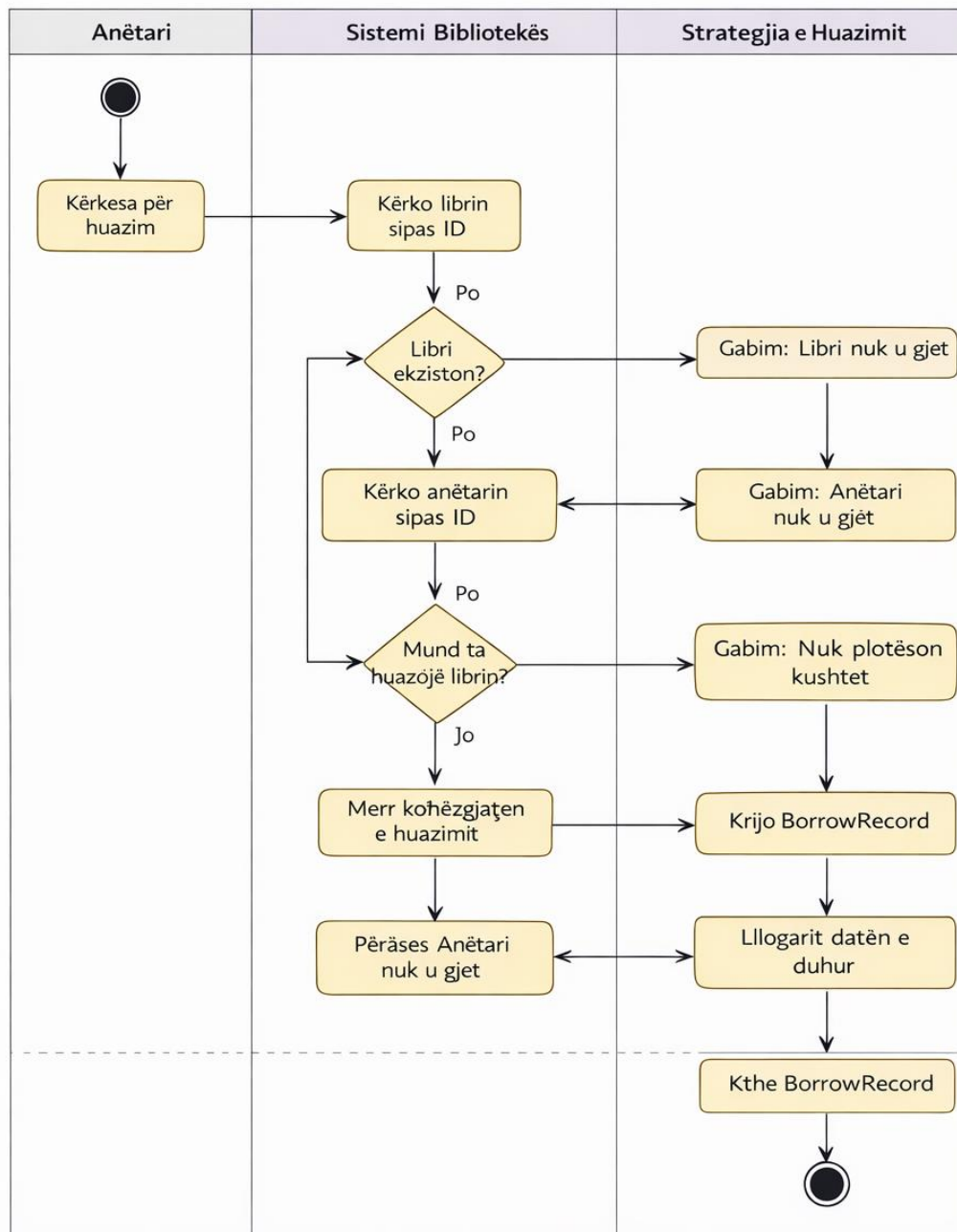


Diagram 5. Diagrami i aktivitetit të sistemit.

7. Arkitektura dhe parimet e dizajnit

Sistemi i Menaxhimit të Bibliotekës është zhvilluar mbi një **arkitekturë të pastër me shtresa të ndara**, e cila ka për qëllim të ndajë qartë përgjegjësitë midis komponentëve të ndryshëm, të minimizojë lidhjet e forta midis tyre, të lehtësojë testimin dhe të sigurojë mundësi të zgjerimit në të ardhmen. Kjo arkitekturë mundëson që secila shtresë të përqendrohet në një aspekt specifik të sistemit dhe që ndryshimet të kryhen me ndikim minimal në pjesët e tjera të kodit.

7.1. Shtresat Kryesore të Arkitekturës

1. Application Entry Point (app.ts)

Ky është pika hyrëse e aplikacionit. Në këtë shtresë krijohen të gjitha instancat e shërbimeve, use case-ve dhe strategjive të huazimit, të cilat më pas injektohen në controller-e përmes **Dependency Injection**. Kjo metodë siguron lidhje të dobëta midis komponentëve dhe kontroll të plotë mbi lifecycle-in e objekteve.

2. Controller Layer

Controller-et përfaqësojnë hyrjen e sistemit për përdoruesit, duke menaxhuar kërkesat HTTP dhe përgjigjet. Ato nuk përmbajnë logjikë të biznesit, por thjesht orkestrojnë thirrjet te use case-t.

- BookController: Menaxhon kërkesat për librat, si krijimi dhe listimi.
- MemberController: Menaxhon kërkesat për anëtarët e bibliotekës.
- BorrowController: Menaxhon proceset e huazimit dhe kthimit të librave.

3. Use Case Layer

Kjo shtresë orkestron logjikën e biznesit duke kombinuar shërbime dhe strategji për të realizuar funksionalitete specifike.

- CreateBookUseCase, GetAllBooksUseCase
- CreateMemberUseCase, GetAllMembersUseCase
- BorrowBookUseCase, ReturnBookUseCase

Çdo use case implementon interface-in `IUseCase<TInput, TOutput>` dhe përmban metodën `execute`, duke siguruar që çdo veprim biznesi të jetë i qartë dhe i izoluar.

4. Service Layer

Kjo shtresë përmban logjikën e biznesit që mund të ripërdoret dhe menaxhimin e të dhënave:

- BookService dhe MemberService për menaxhimin e entiteteve.
 - Strategjitë e huazimit (StandardBorrowingStrategy, PremiumBorrowingStrategy, StudentBorrowingStrategy) përcaktojnë rregullat për huazimin.
- Klasat trashëgojnë nga BaseLibraryService<T> dhe implementojnë interface-t e repository-ve për të siguruar kontrata të qarta dhe standarde përdorimi.

5. Domain Layer

Kjo është shtresa më themelore që përfshin entitetet, interface-t dhe klasat abstrakte:

- **Entities:** Book, Member, BorrowRecord përfaqësojnë të dhënat kryesore të sistemit dhe atributet e tyre.
- **Interface:** IBookRepository, IMemberRepository, IBorrowingStrategy, IUseCase<TInput, TOutput> përcaktojnë kontratat që duhet të respektohen nga shërbimet dhe use case-t.
- **Abstract Classes:** BaseLibraryService<T> ofron strukturë bazë për shërbimet, duke përfshirë metodat e përbashkëta si getAll dhe add dhe metodën abstrakte validate që kërkon implementim specifik në klasat trashëguese.

7.2 Parimet e Programimit Orientuar në Objekte (OOP)

Arkitektura demonstroi përdorimin e **të gjitha parimeve kryesore të OOP**, duke e bërë sistemin modular, fleksibël dhe të ripërdorshëm.

1. **Abstraksioni**

Klasat abstrakte dhe interface-t ofrojnë një përfaqësim të përgjithshëm të komponentëve, duke fshehur detajet e implementimit. Për shembull, `BaseLibraryService<T>` përcakton një metodë abstrakte `validate()` dhe struktura e përgjithshme për menaxhimin e entiteteve.

2. **Trashëgimia**

`BookService` dhe `MemberService` trashëgojnë nga `BaseLibraryService`, duke marrë funksionalitetin e përgjithshëm dhe duke implementuar metodat abstrakte sipas nevojës për secilin entitet.

3. **Polimorfizmi**

Strategjitë e huazimit implementojnë `IBorrowingStrategy`. Use case-i `BorrowBookUseCase` përdor interface-in pa e ditur tipin konkret të strategjisë, duke mundur ndryshim dinamik të sjelljes së huazimit.

4. **Mbishkrimi i Metodave (Overriding)**

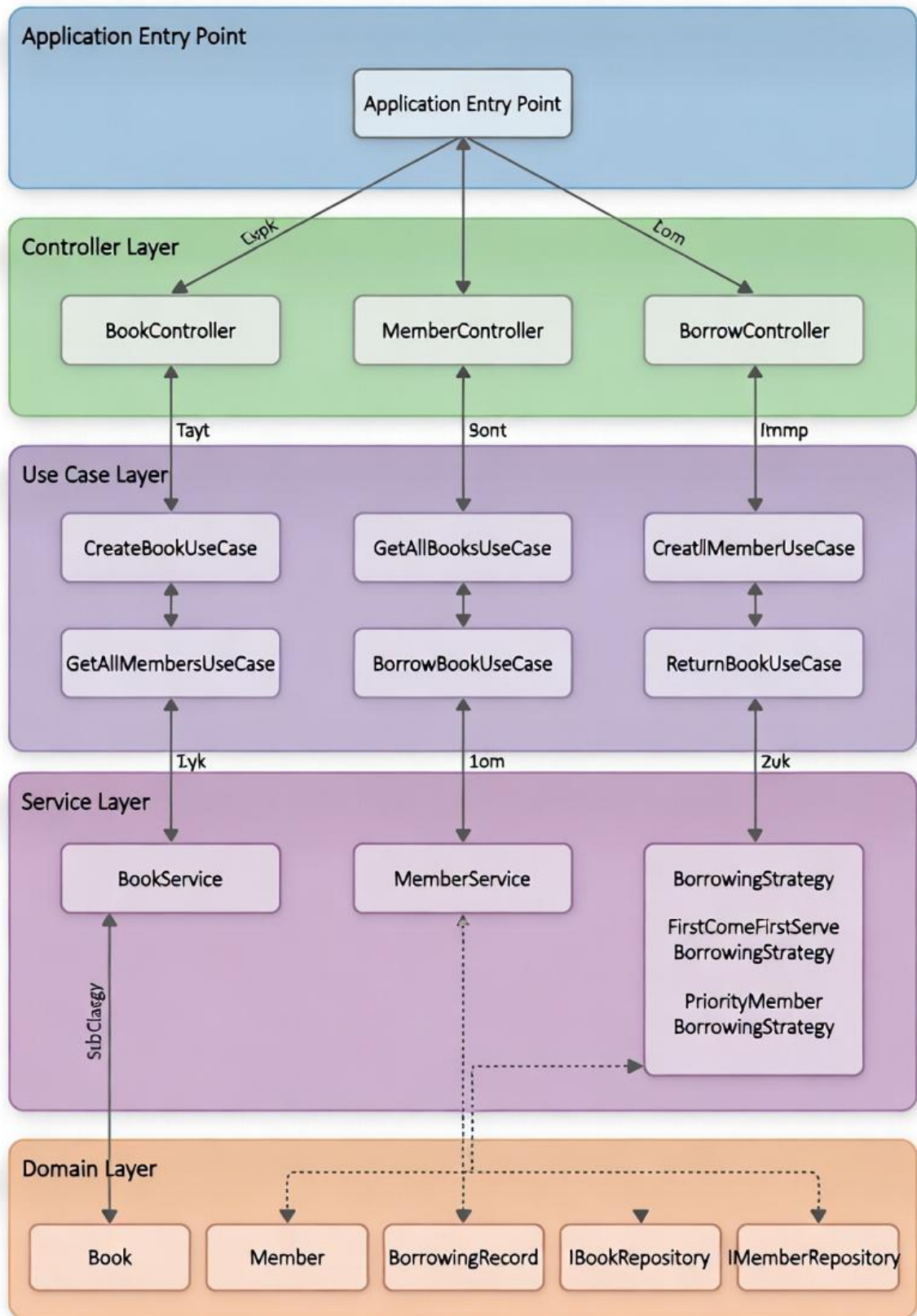
Metoda `validate()` në `BookService` dhe `MemberService` mbishkruhet për të aplikuar logjikë specifike për secilin entitet.

5. **Mbingarkimi i Metodave (Overloading)**

`BookService.findById()` ka dy nënshkrime për fleksibilitet: me një parametër (id) ose me dy parametra (id dhe title).

6. **Interface-t**

Interface-t ofrojnë kontrata të qarta për shërbimet dhe use case-t, duke lehtësuar zgjerimin dhe testimin e sistemit.



Diagrami 6. Dizajni i Arkitekturës me shtresa të sistemit.

7. Implementimi dhe përdorimi

Sistemi është dizajnuar sipas parimeve të Clean Architecture dhe Dependency Inversion, ku çdo shtresë ka përgjegjësi të qarta dhe varësitë rrjedhin nga jashtmi në brendësi, duke siguruar modularitet, testueshmëri dhe mirëmbajtje të lehtë.

BookService trashëgon nga BaseLibraryService<Book> dhe implementon IBookRepository. Ai siguron validimin e të dhënave për titull, autor, vit botimi, kopje dhe ISBN, dhe mbështet kërkime specifike si findByAuthor() dhe mbi-ngarkim të metodës findById().

Për huazimin e librave përdoret Strategy Pattern, ku strategjitë si PremiumBorrowingStrategy përcaktojnë rregullat e huazimit dhe gjobat për vonesat, duke mundësuar ndryshim fleksibël sipas tipit të anëtarit. Use case-t si BorrowBookUseCase orkestrojnë logjikën duke injektuar varësitë dhe duke aplikuar rregullat e strategjisë, duke siguruar izolim të logjikës së biznesit dhe përgjegjësi të qarta për secilin komponent.

Përfitimet kryesore:

- Modularitet i lartë: shtimi i funksionaliteteve bëhet pa ndërhyrje në kod ekzistues.
- Testueshmëri e thjeshtë: secila shtresë testohet veç e veç.
- Mirëmbajtje minimale: ndryshimet në rregulla prekin vetëm një vend.
- Zgjerueshmëri: kalimi në databaza si MongoDB ose PostgreSQL kërkon vetëm ndryshim të repository-ve.

8. Konkluzioni

Sistemi i menaxhimit të bibliotekës përfaqëson një zbatim të qartë të parimeve të **Clean Architecture**, **SOLID** dhe **Dependency Inversion**, duke siguruar që çdo shtresë të ketë përgjegjësi të qartë dhe që varësitë të rrjedhin nga jashtmi në brendësi. Kjo qasje lejon modularitet të lartë, mirëmbajtje të lehtë dhe testueshmëri të pavarur për secilën komponentë.

Komponentët kryesorë, si **BookService**, ofrojnë validim të të dhënave dhe kërkime fleksibël, ndërsa përdorimi i **Strategy Pattern** për huazimin e librave siguron që rregullat e biznesit të jenë të ndryshueshme dinamiskisht sipas tipit të anëtarit. Use case-t, si BorrowBookUseCase, orkestrojnë logjikën duke injektuar varësitë dhe duke aplikuar rregullat specifike, duke mbajtur çdo pjesë të sistemit të izoluar dhe me përgjegjësi të vetme.

Ky model e bën sistemin të qëndrueshëm, i testueshëm dhe i zgjerueshëm, duke mundësuar lehtësisht shtimin e funksionaliteteve të reja si kategori librash, rezervime, raportime dhe integritet me API të jashtme, pa ndërhyrë në kodin ekzistues. Përfitimet kryesore përfshijnë modularitet të lartë, mirëmbajtje minimale, testueshmëri të thjeshtë dhe zgjerueshmëri të lehtë, duke e bërë projektin një bazë solide për zhvillime dhe përdorim afatgjatë.

Në përmbledhje, ky sistem tregon një balancë të fortë mes simplicitetit, fleksibilitetit dhe rigorozitetit të arkitekturës, duke krijuar një platformë të qëndrueshme dhe të gatshme për zgjerime të ardhshme dhe nevoja reale operative të bibliotekës.